算法一：

设 f[i]表示从首都出发到达 i 号城市最少花费的天数，易得转移：

$$f[i] = \mathrm{Max}\{f[j] + |P[i] - P[j]| * B[j] \mid j \in [i-1, i-k]\} + A[i]$$

暴力 DP 即可。

时间复杂度：$O(N^2)$。

期望得分：20 分。

算法二：

另有 10%的数据满足：1<=M<=501。

设 g[k]表示当前满足 P[j]=k 最小的 f[j]，转移时枚举 k 并用 g[k]转移。

时间复杂度：$O(NM)$。

期望得分：30 分（结合算法一）。

算法三：

另有 10%的数据满足：对于所有 1<=i<=N，满足 B[i]=1。

分类讨论 P[i]和 P[j]的大小关系，使用平衡树可以将转移复杂度降为 $O(\log N)$。

时间复杂度：$O(N\log N)$。

期望得分：40 分（结合算法一、二）。

算法四：

另有 20%的数据满足：对于所有 2<=i<=N，满足 P[i-1]<=P[i]且 B[i-1]<=B[i]。

转移方程简化为：

$$f[i] = \mathrm{Max}\left\{\begin{matrix} f[j] - P[j] * B[j] \\ + P[i] * B[j] \end{matrix} \middle| j \in [i-1, i-k]\right\} + A[i]$$

将$(-B[j], f[j] - P[j] * B[j])$作为平面上的点，答案即为一条过该点且斜率为 P[i]的直线的截距，使用斜率优化可将复杂度降为 $O(N)$。

时间复杂度：$O(N)$。

期望得分：60 分（结合算法一、二、三）。

算法五：

另有 20%的数据满足：k=N。

分类讨论 P[i]与 P[j]的大小关系，现仅讨论 P[i]>=P[j]的情况，P[i]<P[j]的情况类似：

$$f[i] = \mathrm{Max}\{f[j] - P[j] * B[j] + P[i] * B[j] \mid P[i] \geq P[j]\} + A[i]$$

设直线 $y = (B[j]) * x + (f[j] - P[j] * B[j])$ $(x \in [P[j], m])$，答案即为直线在 $x = P[i]$时的取值。考虑使用线段树来维护这些直线，用懒标记在每个节点上标记一条覆盖整个区间的直线。

维护两条直线的并时，先计算出两条直线左右端点的值，如果存在一个直线整体在另一条直线下方的情况，直接保留最优直线即可。否则计算两条直线在端点出的函数值，在本节点保留最优区间长的一条直线，把另一条直线向下传递即可。

每条线段最多拆分成 $O(\log N)$个线段树上的节点，每个节点最多下传 $O(\log N)$次。故总时间复杂度为 $O(N\log^2 N)$。

期望得分：60 分（结合算法一、二、三）。

算法六：

在算法五的基础上拓展，因为加入了 k 的限制需要删除直线，线段树无法很容易完成这一操作，我们使用线段树分治来解决。对于每个城市 j 在另一颗线段树上打上[j+1,j+k]的区间戳。分治的时候按先序遍历线段树，先加入该节点标记的直线，对于其子树的所有节点更新一下答案，最后分别递归左右子树即可。

其实分治后问题离线化，我们也可以按斜率排序后直接维护线段的合并结果，只是细节有些复杂，此处暂不讨论。

时间复杂度：$O(Nlog^3N)$（线段树维护）或 $O(Nlog^2N)$（离线维护）。

期望得分：100 分。

（算法七）：

在算法四的基础上继续拓展，同样分类讨论 P[i]与 P[j]的大小关系，现仅讨论 P[i]>=P[j]的情况，P[i]<P[j]的情况类似：

$$f[i] = Max\{f[j] - P[j] * B[j] + P[i] * B[j] \mid P[i] \geq P[j] \text{且} j \in [i-1, i-k]\} + A[i]$$

同样使用线段树分治解决限制 k 的问题，分治后计算左一半节点对右一半节点的贡献时按 P[i]从小到大排序，使用平衡树动态维护$(-B[j], f[j] - P[j] * B[j])$的凸壳，可以做到$O(Nlog^2N)$的时间复杂度。

期望得分：100 分。